WASDeTT 2013 <wasdett2013@easychair.org> 23. Mai 2013 21:07
To: Michael Perscheid
WASDeTT 2013 notification for paper 1

Dear Michael,

We have been trying to send the reviews using easychair for a few times now. I'm trying again now..

Please find your reviews attached, and use them to improve the final version of your paper. The deadline for uploading the final version is: June 10th 2013

Cheers

Jurgen

---------- REVIEW 1 ----------
PAPER: 1
TITLE: Constraining Timing-dependent Communication for Debugging Non-deterministic Failures
AUTHORS: Tim Felgentreff, Michael Perscheid and Robert Hirschfeld

OVERALL EVALUATION: 1 (accept)
REVIEWER'S CONFIDENCE: 4 (high)

---------- REVIEW ----------
Summary:
Debugging anomalies stemming from asynchronous communication is known to be challenging. The approach proposed by the paper is a practical debugging solution to record and debug HTTP-based communication problematic scenarios.

The recording of events is carried out by ContextS, an appealing solution to define contextual and cross-cutting behavior. All the events occurring between two points in a network communication.
Thanks to a dedicated scheduler, constraints about the communication may be defined to reproduce the error and allow for additional debugging. This step essentially correspond to a test case in which checkpoints of the expected behavior are expressed using #assert:

The refinement of remote data is achieved by having a dedicated scheduler accompanied by a domain-specific debuggers. This debugger offers a visualization of the scheduling of the events and give a partial call tree of the source code when an event is selected.

Review:

I have enjoyed reading this paper. It fits well with the Workshop objective.
I have two main comments, that I am pretty confident the authors will easily address.
- Benchmarking. This is a point that the paper should develop since it repeatedly mentions that event recording has a low overhead without giving any indication.

- Problem description. Section 1 briefly compares two techniques: post-mortem debugging and record-and-replay techniques. Giving more indication on what are the differences between these two would help. Page 2, bullet (b): I find this interesting. Maybe a one line example would help getting this idea straight?

Detailed comments:

Page 3: "An approach to record runtime data over multiple executions" => I though that you are interested in non-deterministic condition. Thus discarding reproducible scenarios.

I like section 2.4. It actually phrases well the scope of your tool.

Suggestions for minor rewritings:

"Section 2 presents our approach with the help of an example." => "Section 2 illustrates our approach"
"Section 3 explains Peek-At-Talk and its implementation." => "Section 3 presents the main lines of Peek-At-Talk and gives an overview of its implementation."
"Section 4 describes related work" => "… presents the related work".
Page 3: the sentence "If both customers …" is too long. Please cut it down into pieces.
Page 5: "in Peek-At-Talk ," => remove the space between "k" and ",".
Page 5: a space is missing between I/O and operations. Same things with I/O and devices. I have the gut feeling that you may want to use the xspace latex package :-) Your macro will then be "I/O\xspace"
Page 5: "We this achieve" => "We achieve this control" ?
Page 7: "The schedulers wrap" => Are we talking about one particular scheduler or many of them? I though you are referring to the scheduler you are actually considering.

---------- REVIEW 2 ----------
PAPER: 1
TITLE: Constraining Timing-dependent Communication for Debugging Non-deterministic Failures
AUTHORS: Tim Felgentreff, Michael Perscheid and Robert Hirschfeld

OVERALL EVALUATION: 1 (accept)
REVIEWER'S CONFIDENCE: 3 (medium)

---------- REVIEW ----------
The submitted paper describes an approach, supported by a tool, to help developers find non-deterministic bugs that are due to network communication differences in distributed web applications. The approach consists of instrumenting an application to trap network communication such that traces can be logged, a mechanism to compare traces (successful and unsuccessful ones) to find the differences that can lead to problems, and a traffic shaper to then make the non-deterministic problem reproducible such that a developer can fix this. The claims are that this approach is lightweight and can be enabled on live systems without causing too much overhead, and allows for live introspection of the system so a developer can get information in order to fix the bug more easily once the problem is identified and reproducible.

The paper is relevant for WASDeTT and the problem is a practical and important one. Fixing non-deterministic bugs like this is indeed hard, and the 'bug report' given as example is dreaded but realistic. The paper is well written and reasonably clear. While the explanation can be followed well, the paper lacked any kind of validation apart from a few sentences on page 5 (end of top paragraph) that mention 26MB of additional storage and 0.01 (unit?) overhead in timing. However, none of the setup or application or networks involved are mentioned.

For a journal submission I think that a validation of the claims should certainly be discussed.

In the given paper I remarked a number of places where I wondered about practical issues that I'm fairly certain can easily occur in practice and will invalidate the claims, the most important one being that the state is copied deeply when ZZZZ; while easy to do in Smalltalk more often than not this suddenly results in far more data being stored and crashes. In this case, given that HTTP sends are being investigated, I can imagine that the HTTP context is involved, which will likely involve the web server, which might well need to dump the data of the whole web server with all of its connections and associated data. Of course I do not know if this is an issue, but if I connect to a live system after tracking down a potential bug the last think I would want is the server going down because of my debugging. If this is not an issue then the paper should explain this.

Moreover I now found that the discussion of the actual tool itself was relatively weak, in the sense that there is no actual discussion on alternative approaches (what else could be used besides context-oriented programming ? Would it be an option to instrument not the application but the web server, that has much of the information you need for tracing and replaying and might be able to lift some of the limits mentioned in section 2.4?), lessons learned while making the tool (did you try some alternative implementations that did not work as well as what you currently use?), or some idea of the complexity of the tool.

Some smaller remarks:
- page 5, unit missing in 0.01 (seconds? factor? percent?)
- page 5, Schedule Shaper, paragraph 2: … that API is treated according *to?* the recorded schedules
- page 5, Schedule Shaper, paragraph 2: do single-threaded servers still exist? What if they are multi-threaded?
- page 6, last paragraph before section 2.3: 'assuming otherwise deterministic server' : is this a realistic assumption? I have no feeling in how far this is really the case, but due to multithreading, load balancing, caching, I can imagine that this assumption would not (always) hold in practice…
- page 6, last paragraph before section 2.3: 'will exhibit fewer anomalies …' : this is indeed what a validation should

test on some examples. How many traces are needed to make it useful? How large are these traces ? How many false positives does a developer see ? If the tool shows too many trace differences that have to be inspected more work will be needed to find the 'interesting' differences to focus on.
- page 7: before section 2.4: 'deep copies of the argument'. In a Smalltalk, with many connections between objects, this can quickly explode. In this case I'm afraid that method arguments will easily involve the HTTP request, which probably includes a 'context' argument, which likely includes a reference to the server, request queue or Smalltalk root object and before you know it about the whole machine state has to be dumped. Since this is on a live production system, one of the claims of this paper, it might well bring down the whole system. Some more information on whether this can happen (or not) or some control mechanism such that developers can control this would be appreciated. Maybe some clever approaches can be used to avoid making the deep copies ?
- page 7, section 2.4, second paragraph, closed-world case: I do not see how you can replay events in a live system: the recorded events will be from another (previous) interaction so I doubt you can just use them in a new interaction.
- page 10. top paragraph, 'They also require either a custom vm or additional dynamic libraries that cannot be deactivated at runtime': it seems to me that you leverage the built-in reflection in a Smalltalk vm which also cannot be deactivated at runtime.

---------- REVIEW 3 ----------
PAPER: 1
TITLE: Constraining Timing-dependent Communication for Debugging Non-deterministic Failures
AUTHORS: Tim Felgentreff, Michael Perscheid and Robert Hirschfeld

OVERALL EVALUATION: 0 (unknown)
REVIEWER'S CONFIDENCE: 3 (medium)

---------- REVIEW ----------
Summary

This paper presents the Peek-At-Talk debugging tool for detecting non-deterministic failures due to time-dependent network communication. This tool is based on a "record and refinement" debugging approach:
- recording of the network communication schedules,
- anomaly analysis based on both an abstract model of the communications and the low-level code implementation,
- replay of the failing communication schedule to test dynamically the system using schedule shaper techniques.

The authors illustrate their approach using a flight booking service. In comparison with the state of the art, the main contributions are a recording technique with low overhead (allowing continuous verification) and a refinement mechanism allowing the developer to incrementally select runtime data necessary for debugging the live system.

Remarks

The debugging tool presented in this paper seems promising. However the authors do not really follow the instructions of the WASDeTT workshop. Indeed this tool seems quite new and thus there is no realistic case studies and no discussion about tool design and usability. Instead the authors focus on the presentation of their debugging approach and do not detail the implementation of the underlying tool (the functionalities of the tool are just briefly presented in Section 3). The authors should at least detail Section 3 with design rationale and maybe discuss the benefits of implementing this technique with a language like Smalltalk.

Typos

- p4: the Web servers responds

- p5: I/O: no space after (call \io{} instead of \io)
- p5: We this achieve...

---------- REVIEW 4 ----------
PAPER: 1
TITLE: Constraining Timing-dependent Communication for Debugging Non-deterministic Failures
AUTHORS: Tim Felgentreff, Michael Perscheid and Robert Hirschfeld

OVERALL EVALUATION: 1 (accept)
REVIEWER'S CONFIDENCE: 4 (high)

---------- REVIEW ----------
Overall this looks like a good paper. The debugging technique looks
to be useful and novel, and the implementation seems reasonable.
Two issues: there isn't enough in the paper to really understand what
a "diagnosis script" is, and it would be great to have a larger case study
or other study of the use of the tool.

The assumption that servers are deterministic seems reasonable,
especially since I think you could wrap any other sources of
nondeterminism and record them too. Perhaps you could mention
how the Context programming helps in this? That is, how much
effort is it if another source of non-determinism is discovered?

Some more evaluation would be useful: how long does it take
to get the results of a query to the execution state?

Abstract

"the latter require" => "the latter requires"
(in the same sentance, what does "it" refer to? awkward english!)

avoid "we" unless it is something *you* actually do.
You don't record the network traffic. You write a program
that records network traffic.
"we record…" ==> "Peek-At-Talk records…"

Introduction

You haven't really *established* the truth of limitation (2).

Same issue with "we" in last paragraph on page 2

"less seats" => "fewer seats"

Section 2.1

(Same issue with "we")

Add space after "I/O". THe \xspace package might help you :-)

"constrain the system to such" ==> "constrain the system to those"

What is the relationship between "traffic shaper" and scheduler?

Section 2.2

The description of a diagnosis script is very confusing!
I dont' understand how mimicing a customer helps. Please be more
precise. What happens if the action performs a mutation?

I (think I) understand LCS but your description is confusing

If two sequences differ, how can the LCS be the entire string
(the full schedule)?

Section 2.3

In this case the use of "we" is particularly confusing.
"we add a query". Do you mean the system does it, or the
person doing the debugging?

I suppose you cannot just "set a breakpoint" because it's
a live system? What about debugging an operation that actually
has a transactional effecton the live system? You can't
rerun this over and over, can you?
Do you actually post the same initial HTTP request again,
and then control the resulting actions to be atomic?
A discussion of REST might help here.

I think when you use "record and replay" as an ajective,
you need to put hyphens in:
  "an application-level record-and-replay approach"